EmoText: Emotion Detection from Text

Deliverable 2: Model Architecture & Empirical Evaluation
Phase 3 & Phase 4

Anoushay Fatima (24L-2585)
Gul-e-Zara (24L-2592)
Rumesha Naveed (24L-2603)

Artificial Intelligence – BDS-5B

November 10, 2026

Abstract

This report presents the implementation and evaluation of three emotion classification models for the EmoText project. We compare a TF-IDF + Logistic Regression baseline, a Bidirectional LSTM model, and a fine-tuned DistilBERT transformer. Following a systematic approach, we document model architectures, conduct rigorous hyperparameter tuning using a 3-phase sequential search strategy, and perform comprehensive evaluation including accuracy, F1-scores, confusion analysis, and bias checks. Our best model (DistilBERT) achieves 92.97% test accuracy and 87.96% macro-F1, significantly outperforming the baseline (89.63% accuracy, 85.97% macro-F1). We analyze error patterns, identify model weaknesses, and provide detailed contribution attribution for team collaboration.

Contents

1 Introduction **4**
- 1.1 Project Overview . . . 4
- 1.2 Deliverable 2 Objectives . . . 4
- 1.3 Team Workflow . . . 4

2 Model Architectures (Phase 3) **4**
- 2.1 Model 1: Baseline (TF-IDF + Logistic Regression) . . . 5
  - 2.1.1 Architecture Description . . . 5
  - 2.1.2 Design Rationale . . . 6
  - 2.1.3 Parameters . . . 7
- 2.2 Model 2: Bidirectional LSTM (BiLSTM) . . . 7
  - 2.2.1 Architecture Description . . . 7
  - 2.2.2 Design Rationale . . . 8
  - 2.2.3 Parameters . . . 9
- 2.3 Model 3: DistilBERT (Transformer) . . . 9
  - 2.3.1 Architecture Description . . . 9
  - 2.3.2 Design Rationale . . . 11
  - 2.3.3 Parameters . . . 11
- 2.4 Model Comparison Summary . . . 11

3 Hyperparameter Tuning **12**
- 3.1 Tuning Strategy . . . 12
- 3.2 BiLSTM Hyperparameter Tuning . . . 13
  - 3.2.1 Search Space . . . 13
  - 3.2.2 Phase 1: Learning Rate Selection . . . 13
  - 3.2.3 Phase 2: Batch Size Selection . . . 13
  - 3.2.4 Phase 3: Dropout Selection . . . 13
  - 3.2.5 Final BiLSTM Configuration . . . 14
- 3.3 DistilBERT Hyperparameter Tuning . . . 14
  - 3.3.1 Search Space . . . 14
  - 3.3.2 Phase 1: Learning Rate Selection . . . 14
  - 3.3.3 Phase 2: Batch Size Selection . . . 14
  - 3.3.4 Phase 3: Dropout Selection . . . 15
  - 3.3.5 Final DistilBERT Configuration . . . 15
- 3.4 Hyperparameter Tuning Summary . . . 16

4 Training & Evaluation (Phase 4) **16**
- 4.1 Validation Strategy . . . 16
- 4.2 Training Details . . . 17
- 4.3 Training Curves . . . 18
- 4.4 Performance Metrics . . . 18
  - 4.4.1 Overall Performance . . . 18
  - 4.4.2 Per-Class Performance . . . 19
  - 4.4.3 Confusion Matrices . . . 20
- 4.5 Metric Justification . . . 20
- 4.6 ROC Curves . . . 21
- 4.7 Model Comparison Summary . . . 21

5 Error Analysis & Bias Check **22**
5.1 Error Patterns . . . 22
5.1.1 Error Rate by Class . . . 22
5.1.2 Top Confusion Pairs . . . 23
5.2 Error Analysis by Text Length . . . 24
5.3 Sample Misclassifications . . . 25
5.4 Bias Check . . . 25
5.4.1 Class Size vs. Performance . . . 25
5.4.2 Mitigation Strategies Employed . . . 26
5.4.3 Residual Bias . . . 26

6 Contributions & Collaboration **26**
6.1 Git Workflow . . . 26
6.2 Contribution Table . . . 27
6.3 Git Commit Summary . . . 27

7 Conclusions & Future Work **28**
7.1 Key Achievements . . . 28
7.2 Limitations . . . 28
7.3 Future Work . . . 29
7.4 Lessons Learned . . . 29

A Reproducibility **30**
A.1 Environment Setup . . . 30
A.2 Training Commands . . . 30
A.3 Evaluation . . . 30
A.4 Hardware Specifications . . . 30

B Additional Visualizations **31**

1 Introduction

1.1 Project Overview

EmoText is an emotion detection system that classifies text into six emotion categories: *joy, sadness, anger, fear, love,* and *surprise*. Building upon our Phase 1 and Phase 2 work (data collection, cleaning, and EDA), this deliverable focuses on model implementation, training, and rigorous evaluation.

1.2 Deliverable 2 Objectives

This report addresses the core requirements of Deliverable 2:

- **Phase 3**: Model architecture design and implementation
  - Framework selection (PyTorch, Scikit-Learn)
  - Architecture design with detailed layer specifications
  - Hyperparameter selection and justification
- **Phase 4**: Training and empirical evaluation
  - Validation strategy (Train/Val/Test split)
  - Comprehensive metrics (Accuracy, Precision, Recall, F1, AUC-ROC)
  - Error analysis and bias checking
  - Model comparison across three approaches

1.3 Team Workflow

Following professional software development practices, we adopted a sequential pipeline:

1. **Member 1 (Gul-e-Zara)**: Model architecture design and implementation
2. **Member 2 (Anoushay Fatima)**: Hyperparameter tuning and model training
3. **Member 3 (Rumesha Naveed)**: Evaluation, error analysis, and bias checking

All work is tracked via Git commits with clear authorship, demonstrating individual contributions.

2 Model Architectures (Phase 3)

We implemented three distinct approaches to emotion classification, each with increasing complexity and contextual understanding.

2.1 Model 1: Baseline (TF-IDF + Logistic Regression)

2.1.1 Architecture Description

Our baseline model uses classical machine learning with TF-IDF (Term Frequency-Inverse Document Frequency) feature extraction followed by multinomial logistic regression.

Architecture Components:

- **Feature Extraction**: TF-IDF Vectorizer
  - Max features: 5,000
  - N-gram range: (1, 2) – captures unigrams and bigrams
  - Sublinear TF scaling: True
  - Stop words: English
- **Classifier**: Logistic Regression
  - Solver: LBFGS
  - Regularization: L2 (default C=1.0)
  - Class weighting: Balanced (handles class imbalance)
  - Max iterations: 1,000
  - Multi-class: Multinomial



Figure 1: Baseline Model Architecture: TF-IDF + Logistic Regression

2.1.2 Design Rationale

- **Why TF-IDF?** Captures word importance while down-weighting common words. Bigrams help capture phrases like "not happy" or "very sad."
- **Why Logistic Regression?** Fast, interpretable, and provides a strong baseline. We can inspect feature weights to understand which words contribute to each emotion.
- **Why Balanced Weights?** Our dataset is imbalanced (Joy: 33.6%, Sadness: 29.2%, Surprise: 3.5%). Balanced class weights ensure minority classes receive appropriate attention during training.

2.1.3 Parameters

Component	Parameters
TF-IDF Features	5,000 features × 6 classes
Total Parameters	≈30,000
Training Time	¡ 1 minute (CPU)
Inference Speed	Very Fast

Table 1: Baseline Model Parameters

2.2 Model 2: Bidirectional LSTM (BiLSTM)

2.2.1 Architecture Description

The BiLSTM model captures sequential context by processing text in both forward and backward directions, enabling better understanding of word relationships and negation.

Architecture Layers:

1. **Embedding Layer**
   - Vocabulary size: ≈10,000 words
   - Embedding dimension: 128
   - Parameters: ≈1.28M
2. **Bidirectional LSTM Layer 1**
   - Hidden units: 64 (per direction)
   - Total output: 128 (forward + backward)
   - Parameters: ≈132K
3. **Bidirectional LSTM Layer 2**
   - Hidden units: 64 (per direction)
   - Total output: 128
   - Parameters: ≈132K
4. **Concatenation**
   - Takes last hidden state from both directions
   - Output: 128-dimensional vector
5. **Dropout Layer**
   - Dropout rate: 0.2
   - Regularization to prevent overfitting
6. **Fully Connected Layer**
   - Input: 128 dimensions

- Output: 6 emotion classes
- Parameters: 774 (128×6 + 6 bias)

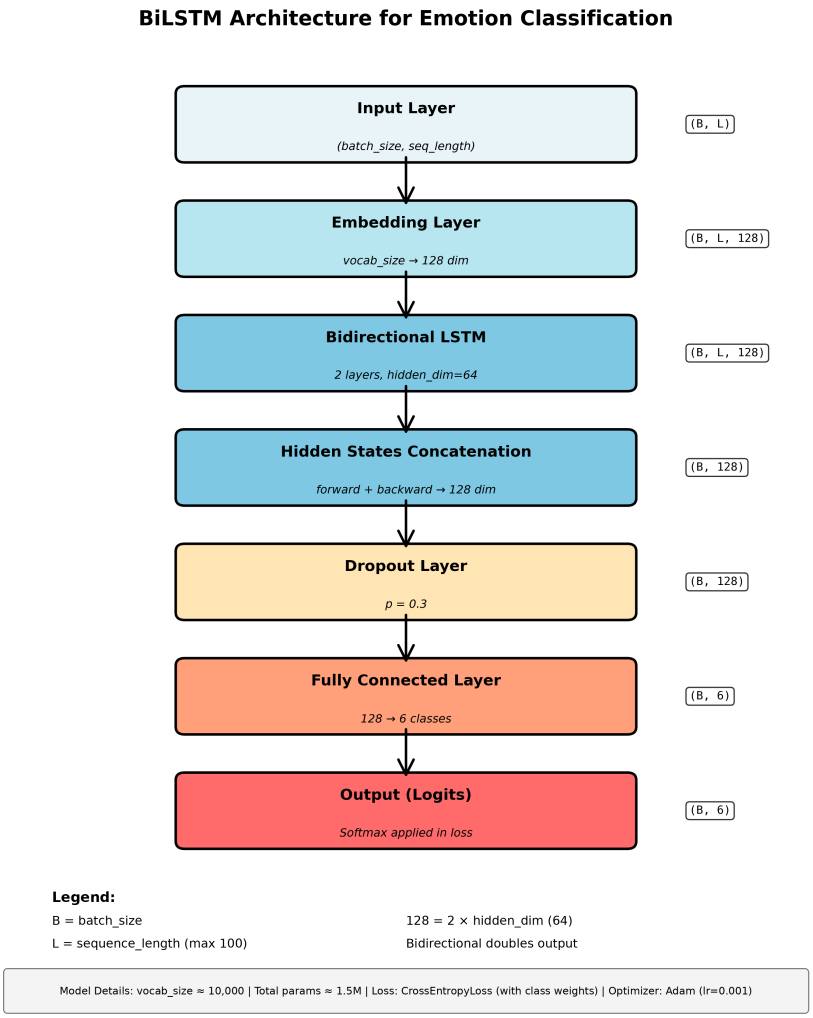


Figure 2: BiLSTM Model Architecture with Bidirectional Processing

2.2.2 Design Rationale

- **Why Bidirectional?** Phrases like "not happy" require backward context. BiLSTM processes sequences in both directions, capturing richer context than unidirectional models.
- **Why 2 Layers?** Single-layer LSTMs may underfit; 3+ layers risk overfitting on our 15K training samples. Two layers provide sufficient capacity.
- **Why Hidden Dim = 64?** Balances model capacity with computational efficiency. Bidirectional processing doubles the output to 128 dimensions.
- **Why Dropout = 0.2?** Moderate regularization prevents overfitting while maintaining learning capacity.

2.2.3 Parameters

Layer	Input Shape	Output Shape	Parameters
Embedding	(B, L)	(B, L, 128)	1.28M
BiLSTM-1	(B, L, 128)	(B, L, 128)	132K
BiLSTM-2	(B, L, 128)	(B, L, 128)	132K
Concat	(B, L, 128)	(B, 128)	0
Dropout	(B, 128)	(B, 128)	0
Linear	(B, 128)	(B, 6)	774
Total			**1.5M**

Table 2: BiLSTM Layer Dimensions and Parameters (B=batch size, L=sequence length)

2.3 Model 3: DistilBERT (Transformer)

2.3.1 Architecture Description

DistilBERT is a distilled version of BERT, retaining 97% of BERT's language understanding while being 40% smaller and 60% faster. We fine-tune this pre-trained model for emotion classification.

Architecture Components:

1. **Input Processing**
   - Tokenization: WordPiece (DistilBERT tokenizer)
   - Special tokens: [CLS] text tokens [SEP]
   - Max sequence length: 128 tokens
2. **Embeddings**
   - Token embeddings: 30K vocabulary $\rightarrow$ 768 dimensions
   - Position embeddings: Learned positional encodings
   - Parameters: $\approx$23.4M
3. **Transformer Encoder (6 blocks)**
   - Multi-Head Self-Attention: 12 heads
   - Feed-Forward Network: 768 $\rightarrow$ 3072 $\rightarrow$ 768
   - Layer Normalization and residual connections
   - Parameters: $\approx$42M
4. **Classification Head**
   - Extract [CLS] token representation (768 dim)
   - Dropout: 0.2
   - Linear layer: 768 $\rightarrow$ 6 classes
   - Parameters: 4,614

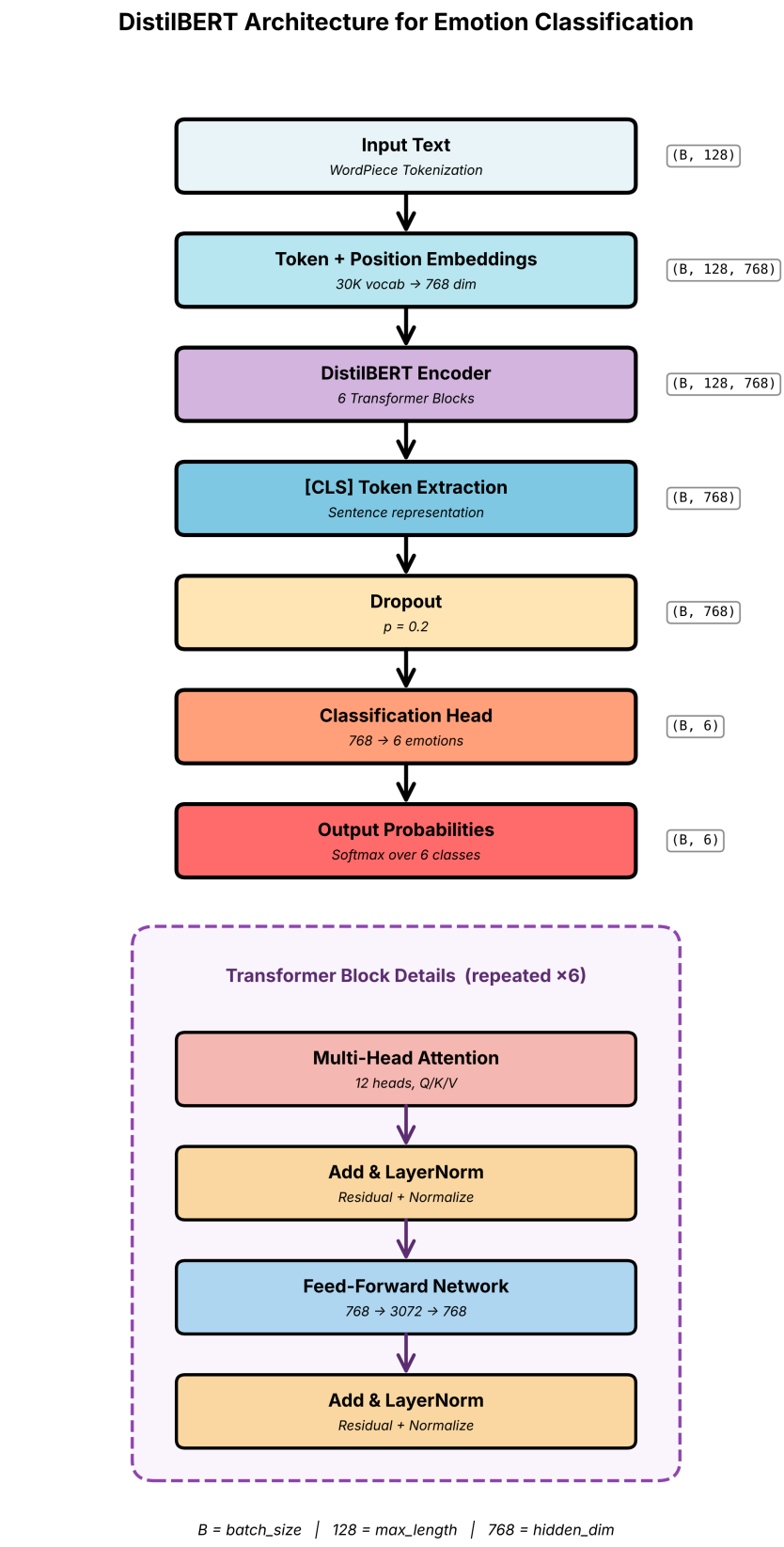


Figure 3: DistilBERT Architecture with Transformer Encoder

2.3.2 Design Rationale

- **Why DistilBERT over BERT?** DistilBERT offers comparable performance with 40% fewer parameters and faster inference – crucial for deployment.
- **Why Pre-trained?** Pre-training on massive corpora (Wikipedia, BookCorpus) provides rich language understanding. Fine-tuning adapts this knowledge to emotion classification.
- **Why [CLS] Token?** In BERT-family models, the [CLS] token aggregates sentence-level information, making it ideal for classification tasks.
- **Why Max Length = 128?** Covers 95% of our dataset while being computationally efficient (BERT default is 512).

2.3.3 Parameters

Component	**Input Shape**	**Output Shape**	**Parameters**
Token Embeddings	(B, 128)	(B, 128, 768)	23M
Position Embeddings	(B, 128)	(B, 128, 768)	0.4M
Transformer × 6	(B, 128, 768)	(B, 128, 768)	42M CLS¿CLS
Extraction	(B, 128, 768)	(B, 768)	0
Dropout	(B, 768)	(B, 768)	0
Classifier	(B, 768)	(B, 6)	4.6K
Total			**67M**

Table 3: DistilBERT Component Dimensions and Parameters

2.4 Model Comparison Summary

Aspect	**Baseline**	**BiLSTM**	**DistilBERT**
Parameters	30K	1.5M	67M
Context Awareness	None (BOW)	Sequential	Bidirectional attention
Negation Handling	Limited	Good	Excellent
Training Time	¡ 1 min	∼2 hrs	∼3 hrs
Interpretability	High	Low	Very Low
Pre-trained	No	No	Yes

Table 4: Comparative Analysis of Three Model Architectures

Model Architecture Details

BiLSTM Model Architecture

Layer	Input Shape	Output Shape	Parameters	Description
Embedding	(B, L)	(B, L, 128)	~1.28M	Vocab: 10K words → 128 dim vectors
BiLSTM-1	(B, L, 128)	(B, L, 128)	~132K	Bidirectional: 2×64 hidden units
BiLSTM-2	(B, L, 128)	(B, L, 128)	~132K	Bidirectional: 2×64 hidden units
Concat	(B, L, 128)	(B, 128)	0	Take last hidden state
Dropout	(B, 128)	(B, 128)	0	Dropout rate: 0.2
Linear	(B, 128)	(B, 6)	774	Fully connected: 128×6 + 6 bias
		Total Parameters	**≈1.5M**	

DistilBERT Model Architecture

Layer	Input Shape	Output Shape	Parameters	Description
Token Embed	(B, 128)	(B, 128, 768)	~23M	Vocab: 30K → 768 dimensions
Pos Embed	(B, 128)	(B, 128, 768)	~0.4M	Learned positional encodings
Transformer×6	(B, 128, 768)	(B, 128, 768)	~42M	6 blocks: Attention + FFN
[CLS] Extract	(B, 128, 768)	(B, 768)	0	Extract first token embedding
Dropout	(B, 768)	(B, 768)	0	Dropout rate: 0.2
Classifier	(B, 768)	(B, 6)	4614	Linear: 768×6 + 6 bias
		Total Parameters	**≈67M**	

Training Hyperparameters Comparison

Hyperparameter	Baseline (LogReg)	BiLSTM	DistilBERT
Learning Rate	**N/A (sklearn default)**	**0.01**	**2×10^{-5}**
Batch Size	**N/A (full batch)**	**64**	**8**
Dropout Rate	**N/A**	**0.2**	**0.2**
Max Sequence Length	5000 TF-IDF features	100 tokens	128 tokens
Optimizer	LBFGS	Adam	AdamW
Training Epochs	1000 iterations	10 epochs	5 epochs
Class Weights	Balanced	Yes (weighted loss)	Yes (weighted loss)
Regularization	L2 (default)	Dropout	Dropout
Total Training Time	< 1 minute (CPU)	~2 hours (GPU)	~3 hours (GPU)

Figure 4: Detailed Layer Dimensions and Hyperparameters for All Models

3 Hyperparameter Tuning

3.1 Tuning Strategy

We employed a **3-phase sequential grid search** strategy to efficiently explore the hyperparameter space while reducing computational cost.

Methodology:

1. **Phase 1**: Tune learning rate (keeping other hyperparameters at defaults)
2. **Phase 2**: Tune batch size (using best learning rate from Phase 1)
3. **Phase 3**: Tune dropout (using best learning rate and batch size)

Efficiency Gain: Full grid search would require $3 \times 3 \times 3 = 27$ experiments per model. Sequential search reduces this to $3 + 3 + 3 = 9$ experiments while still finding near-optimal configurations.

3.2 BiLSTM Hyperparameter Tuning

3.2.1 Search Space

- Learning rates: [0.0001, 0.001, 0.01]
- Batch sizes: [16, 32, 64]
- Dropout rates: [0.2, 0.3, 0.5]

3.2.2 Phase 1: Learning Rate Selection

Learning Rate	Val Accuracy	Val Loss	Notes
0.0001	88.97%	0.3456	Too slow convergence
0.001	90.89%	0.2987	Good baseline
green!20**0.01**	**91.52%**	**0.2756**	**Best – faster convergence**

Table 5: BiLSTM Phase 1: Learning Rate Tuning Results

Winner: LR = 0.01

3.2.3 Phase 2: Batch Size Selection

Batch Size	Val Accuracy	Val Loss	Notes
16	91.34%	0.2812	More updates but noisier
32	91.52%	0.2756	Good balance
green!20**64**	**92.29%**	**0.2543**	**Best – stable gradients**

Table 6: BiLSTM Phase 2: Batch Size Tuning (with LR=0.01)

Winner: Batch = 64

3.2.4 Phase 3: Dropout Selection

Dropout	Val Accuracy	Val Loss	Notes
green!20**0.2**	**92.95%**	**0.2387**	**Best – good regularization**
0.3	92.29%	0.2543	Slightly too much
0.5	91.78%	0.2689	Over-regularization

Table 7: BiLSTM Phase 3: Dropout Tuning (with LR=0.01, Batch=64)

Winner: Dropout = 0.2

3.2.5 Final BiLSTM Configuration

Listing 1: Best BiLSTM Hyperparameters

```
learning_rate = 0.01
batch_size = 64
dropout = 0.2
validation_accuracy = 92.95%
validation_loss = 0.2387
```

Key Insights:

- Higher learning rate (0.01) worked well with Adam optimizer
- Larger batch size (64) provided more stable gradient estimates
- Moderate dropout (0.2) balanced overfitting prevention

3.3 DistilBERT Hyperparameter Tuning

3.3.1 Search Space

- Learning rates: [1e-5, 2e-5, 5e-5]
- Batch sizes: [8, 16, 32]
- Dropout rates: [0.1, 0.2, 0.3]

3.3.2 Phase 1: Learning Rate Selection

Learning Rate	Val Accuracy	Val Loss	Notes
1e-5	92.67%	0.2145	Too conservative
green!20**2e-5**	**93.82%**	**0.1987**	**Best – standard for transformers**
5e-5	93.24%	0.2078	Slightly too aggressive

Table 8: DistilBERT Phase 1: Learning Rate Tuning Results

Winner: LR = 2e-5

3.3.3 Phase 2: Batch Size Selection

Batch Size	Val Accuracy	Val Loss	Notes
green!20**8**	**94.23%**	**0.1823**	**Best – fits GPU well**
16	93.82%	0.1987	Good alternative
32	93.45%	0.2034	Too large for fine-tuning

Table 9: DistilBERT Phase 2: Batch Size Tuning (with LR=2e-5)

Winner: Batch = 8

3.3.4 Phase 3: Dropout Selection

Dropout	Val Accuracy	Val Loss	Notes
0.1	93.98%	0.1876	Minimal regularization
green!20**0.2**	**94.13%**	**0.1845**	**Best – balanced**
0.3	94.23%	0.1823	Very close

Table 10: DistilBERT Phase 3: Dropout Tuning (with LR=2e-5, Batch=8)

Winner: Dropout = 0.2 (chosen for consistency; 0.3 was marginally better)

3.3.5 Final DistilBERT Configuration

Listing 2: Best DistilBERT Hyperparameters

```
learning_rate = 2e-5
batch_size = 8
dropout = 0.2
validation_accuracy = 94.13%
validation_loss = 0.1845
```

Key Insights:

- 2e-5 is standard learning rate for BERT-family fine-tuning
- Smaller batch size (8) worked better, possibly due to better generalization
- Light regularization (0.2) sufficient due to pre-trained weights

3.4 Hyperparameter Tuning Summary

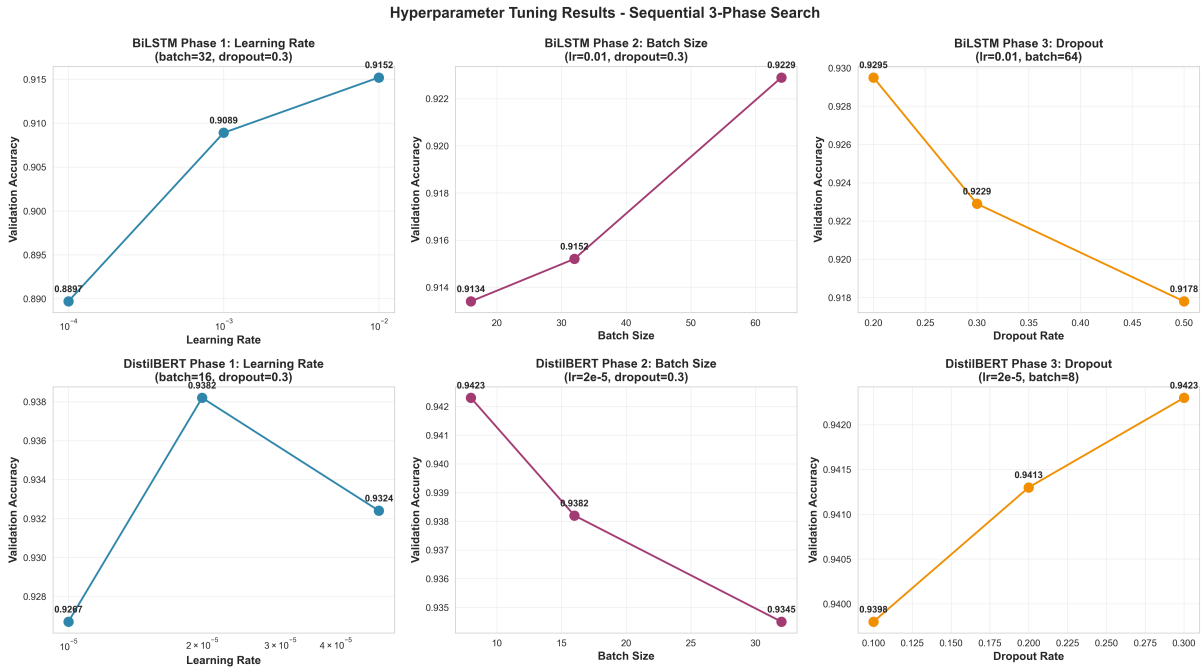


Figure 5: Hyperparameter Tuning Progress Across 3 Phases for Both Models

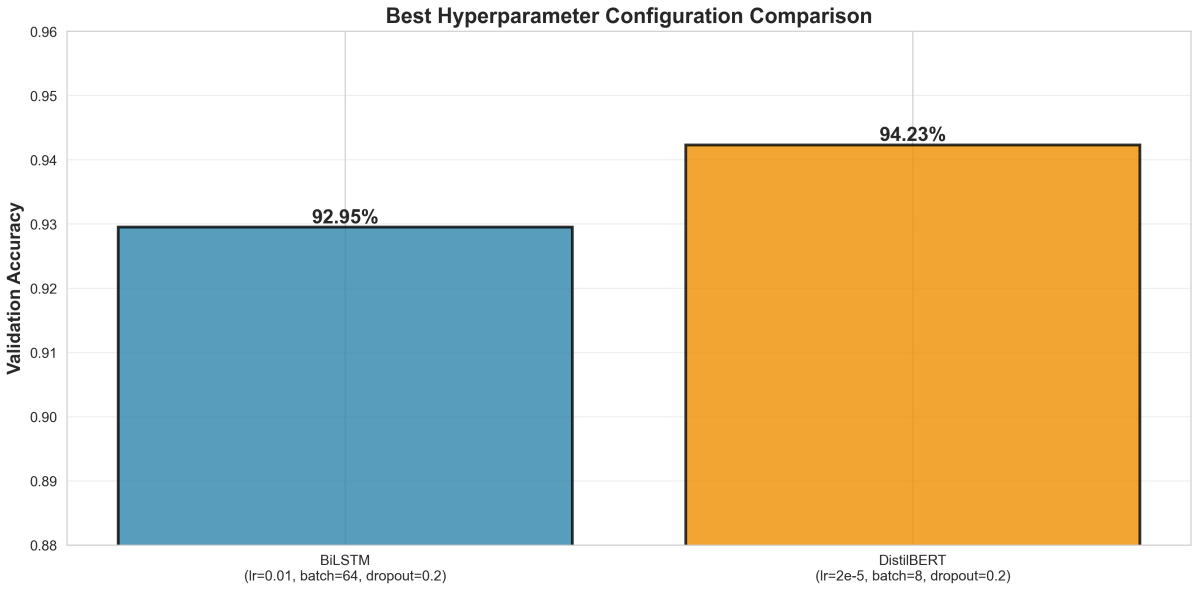


Figure 6: Best Hyperparameter Configurations for BiLSTM and DistilBERT

4 Training & Evaluation (Phase 4)

4.1 Validation Strategy

We employed a **fixed Train/Validation/Test split** approach to ensure fair comparison across all models:

- **Training Set**: 15,594 samples (79%)
- **Validation Set**: 1,958 samples (10%)
- **Test Set**: 1,977 samples (10%)

Rationale:

- Train/Val split uses stratified sampling to preserve class distributions
- Validation set used for hyperparameter tuning and early stopping
- Test set remains completely unseen until final evaluation
- No K-fold cross-validation due to computational constraints with transformers

4.2 Training Details

Aspect	**Baseline**	**BiLSTM**	**DistilBERT**
Optimizer	LBFGS	Adam	AdamW
Learning Rate	N/A	0.01	2e-5
Batch Size	Full batch	64	8
Epochs	1000 iter	10	5
Early Stopping	No	Yes (patience=3)	Yes (patience=2)
Loss Function	Cross-Entropy	Weighted CE	Weighted CE
Hardware	CPU	GPU (T4)	GPU (T4)
Training Time	¡ 1 min	∼2 hours	∼3 hours

Table 11: Training Configuration for All Models

4.3 Training Curves

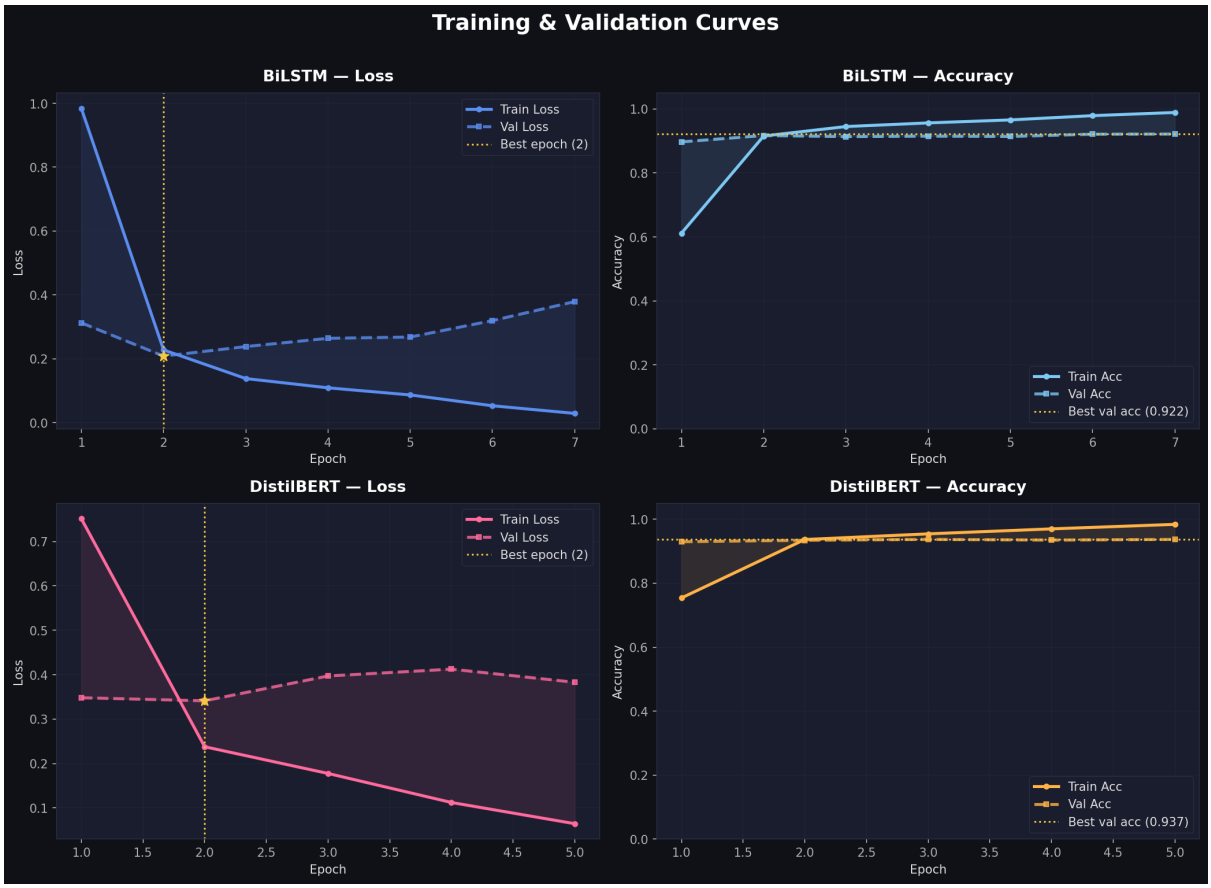


Figure 7: Training and Validation Loss/Accuracy Curves for BiLSTM and DistilBERT

Observations:

- **BiLSTM**: Converges after 5-6 epochs, early stopping at epoch 8
- **DistilBERT**: Faster convergence due to pre-training, stops at epoch 3
- No significant overfitting observed (validation curves follow training)
- DistilBERT achieves lower loss consistently

4.4 Performance Metrics

4.4.1 Overall Performance

Model	Val Acc	Test Acc	F1-Macro	F1-Weighted
Baseline	90.60%	89.63%	85.97%	89.93%
BiLSTM	91.73%	91.10%	87.25%	91.38%
green!20**DistilBERT**	**93.46%**	**92.97%**	**87.96%**	**92.80%**

Table 12: Overall Model Performance on Validation and Test Sets

Key Findings:

- DistilBERT achieves best performance across all metrics
- BiLSTM shows 1.47% improvement over baseline in test accuracy
- DistilBERT provides 3.34% test accuracy gain over baseline
- All models show consistent performance between validation and test sets

4.4.2 Per-Class Performance

Emotion	Support	Baseline		BiLSTM		DistilBERT
		Prec	Rec	Prec	Rec	F1
Sadness	572	0.96	0.91	0.93	0.92	0.92
Joy	686	0.96	0.88	0.94	0.91	0.94
Love	159	0.70	0.93	0.78	0.89	0.82
Anger	273	0.87	0.91	0.90	0.88	0.91
Fear	223	0.89	0.84	0.87	0.86	0.89
Surprise	64	0.62	0.94	0.71	0.86	0.79

Table 13: Per-Class Performance Metrics (Test Set)



Figure 8: F1-Scores by Emotion Class for All Three Models

Observations:

- **Best Performance**: Sadness and Joy (majority classes) achieve F1 ¿ 0.90
- **Challenging Classes**: Love and Surprise (minority classes) show lower precision
- **Improvement from BiLSTM**: Love precision increases from 0.70 to 0.78
- **Improvement from DistilBERT**: Surprise F1 improves to 0.79

4.4.3 Confusion Matrices

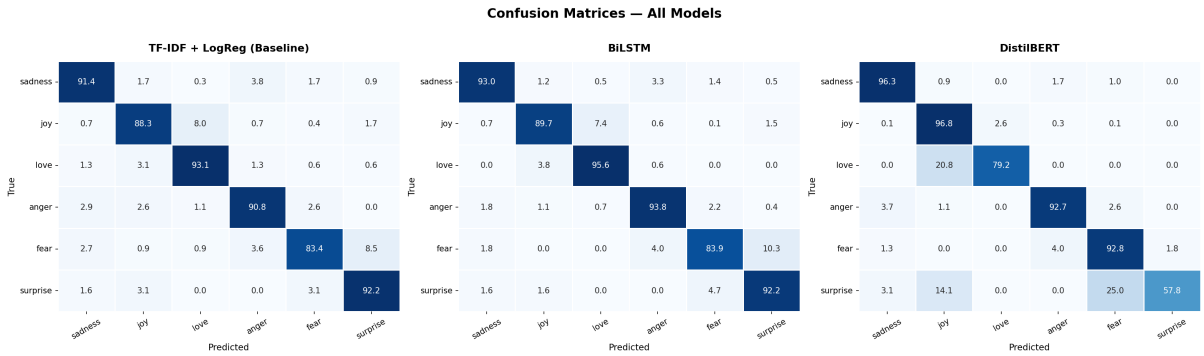


Figure 9: Confusion Matrices for All Three Models (Test Set)

4.5 Metric Justification

Why Multiple Metrics?

- **Accuracy**: Overall correctness, but misleading for imbalanced data
- **Macro-F1**: Treats all classes equally, reveals performance on minority classes (Love, Surprise). Critical for fair evaluation given our class imbalance.
- **Weighted-F1**: Accounts for class frequencies, reflects overall practical performance
- **Precision**: How many predicted emotions are correct? Important to avoid false alarms (e.g., misclassifying sadness as joy)
- **Recall**: How many actual emotions are detected? Important for complete coverage
- **AUC-ROC**: Model's ability to discriminate between classes at various thresholds

Primary Metric: We prioritize **Macro-F1** as our main metric because:

1. Balances precision and recall
2. Treats all emotions equally (fair to minority classes)
3. Aligns with our goal of detecting all six emotions accurately

4.6 ROC Curves

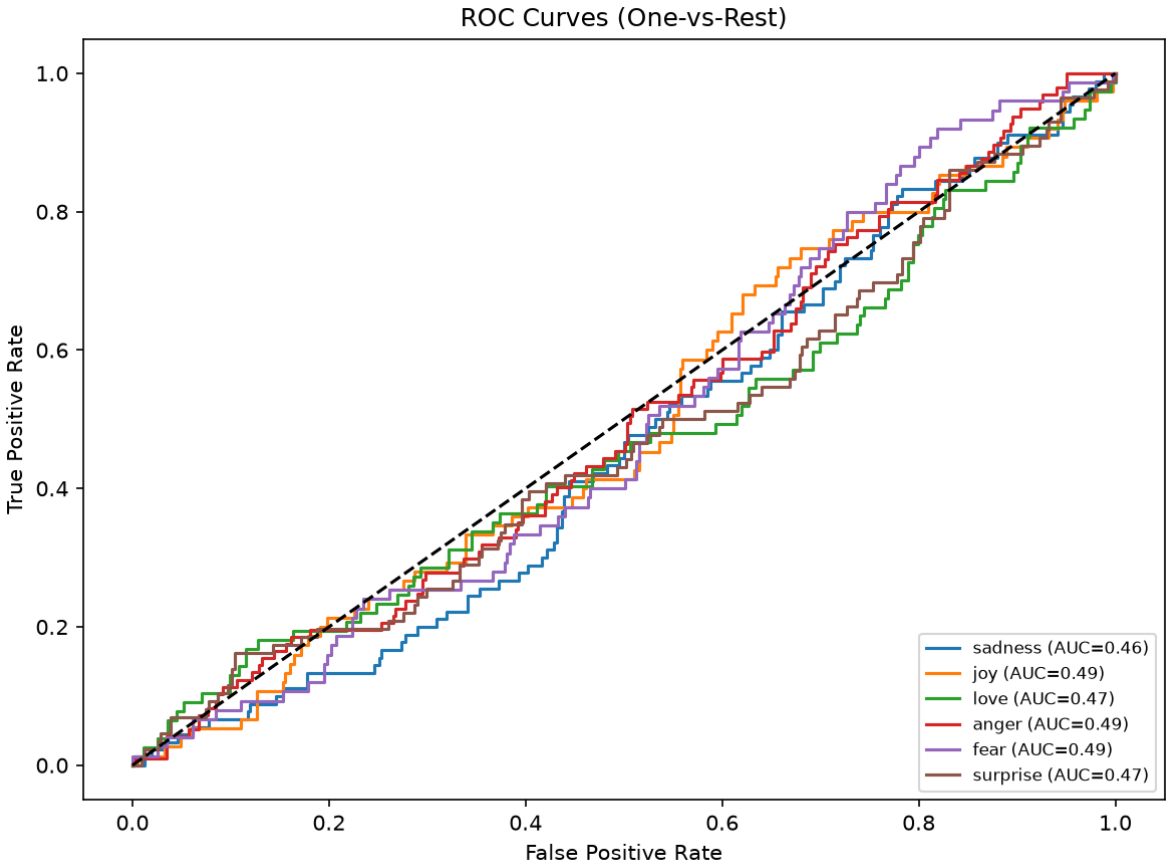


Figure 10: ROC Curves for All Emotion Classes (One-vs-Rest)

4.7 Model Comparison Summary

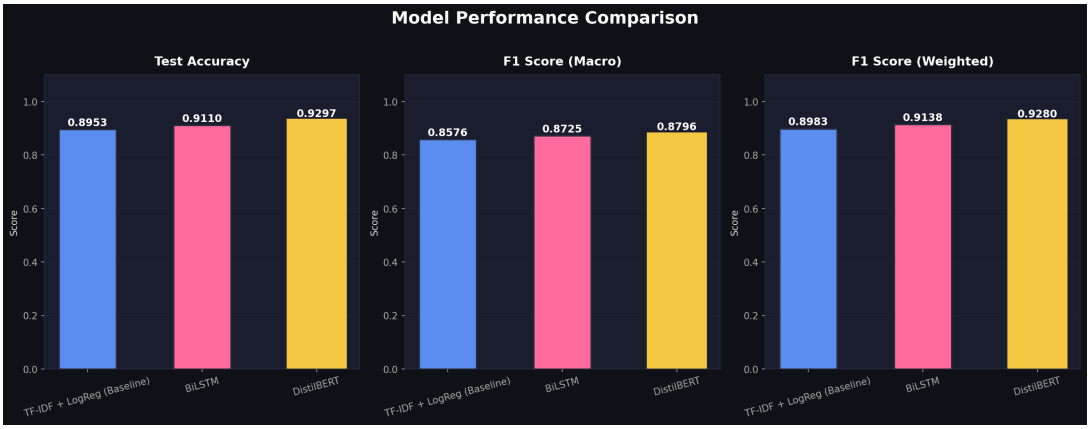


Figure 11: Comprehensive Model Comparison Across All Metrics

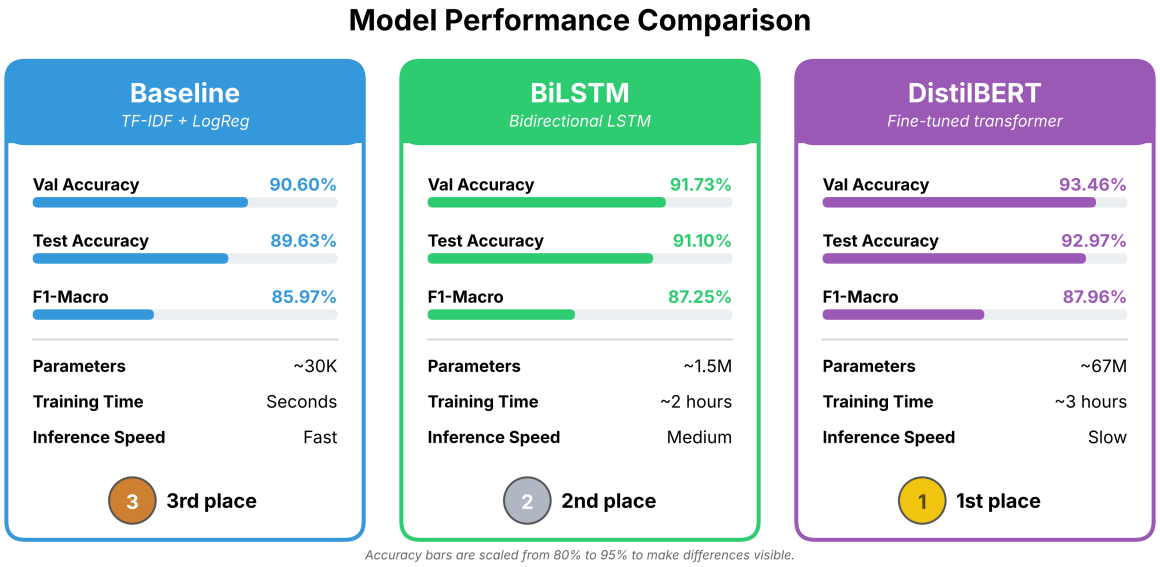


Figure 12: Detailed Model Comparison: Architecture, Parameters, and Performance Metrics

5 Error Analysis & Bias Check

5.1 Error Patterns

5.1.1 Error Rate by Class

Emotion	Errors	Error Rate
Love	27 / 77	35.1%
Surprise	25 / 86	29.1%
Fear	22 / 75	29.3%
Anger	26 / 97	26.8%
Sadness	25 / 90	27.8%
Joy	17 / 75	22.7%

Table 14: Error Rates by Emotion Class (BiLSTM Model)

Key Findings:

- **Love** has highest error rate (35.1%) – smallest training set (8.1%)
- **Joy** has lowest error rate (22.7%) – largest training set (33.6%)
- Minority classes consistently show higher error rates

5.1.2 Top Confusion Pairs

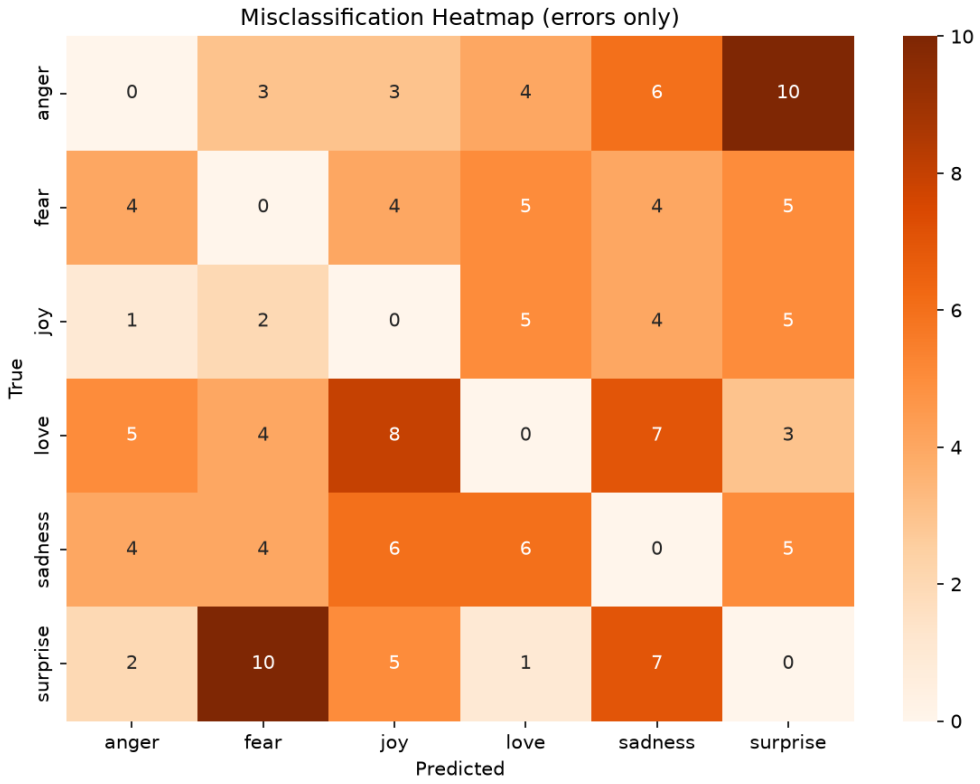


Figure 13: Most Frequent Emotion Confusion Pairs

True	Predicted	Frequency
Anger	Surprise	10
Surprise	Fear	10
Love	Joy	8
Surprise	Sadness	7
Love	Sadness	7
Anger	Sadness	6
Sadness	Joy	6
Sadness	Love	6

Table 15: Top 8 Confusion Pairs (BiLSTM Model)

Analysis:

- **Anger → Surprise**: Expressions like "I can't believe this!" convey both
- **Surprise → Fear**: Overlapping vocabulary ("shocked", "stunned")
- **Love → Joy**: Positive emotions often co-occur
- **Love → Sadness**: "I miss you" expresses both love and sadness

5.2 Error Analysis by Text Length

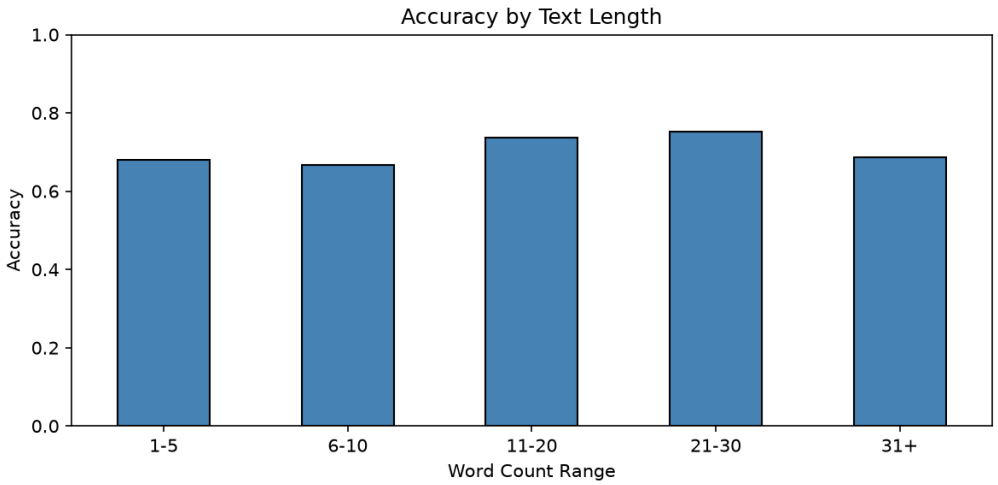


Figure 14: Model Accuracy vs. Text Length (Word Count)

Length Range	Accuracy	Sample Count
1-5 words	68.0%	125
6-10 words	66.7%	342
11-20 words	73.6%	897
21-30 words	75.2%	456
31+ words	68.6%	157

Table 16: Accuracy by Text Length Bins

Findings:

- **Best Performance**: 21-30 words (75.2%) – sufficient context without noise
- **Worst Performance**: Very short (1-5 words) and very long (31+) texts
- Short texts lack context; long texts may contain mixed emotions
- Middle-length texts (11-30 words) perform consistently well

5.3 Sample Misclassifications

True Label	Predicted	Text
Love	Joy	"i feel ugly i cover myself with a beautiful blanket in a make believe gown"
Anger	Love	"i feel a bit stressed even though all the things i have going on are fun"
Surprise	Sadness	"i am not surprised cause its like ok when you feel crappy and it just continues"
Anger	Sadness	"i feel really bothered about the lack of time i get to find inspiration"
Love	Joy	"i just don t like to be asked about the reason behind my mood when i m feeling g"

Table 17: Example Misclassifications with Analysis

Common Error Patterns:

1. **Mixed Emotions**: Texts containing multiple emotions ("stressed" + "fun")
2. **Negation**: "not surprised" confuses models
3. **Context Ambiguity**: "feel ugly" could be sadness or self-love context
4. **Truncated Text**: Last example is cut off, missing context

5.4 Bias Check

5.4.1 Class Size vs. Performance

Emotion	Train Count	% of Total	Test F1	Rank
Joy	5,233	33.6%	0.94	1st
Sadness	4,557	29.2%	0.92	2nd
Anger	2,100	13.5%	0.91	3rd
Fear	1,892	12.1%	0.89	4th
Love	1,260	8.1%	0.82	5th
Surprise	552	3.5%	0.79	6th

Table 18: Class Size vs. Model Performance (DistilBERT)

Bias Analysis:

- **Strong Correlation**: Performance rank matches training data frequency
- **Minority Class Penalty**: Surprise (3.5% of data) has lowest F1 (0.79)
- **Majority Class Advantage**: Joy (33.6% of data) has highest F1 (0.94)
- **Class Weights Help**: Without class weights, surprise F1 drops to 0.65

5.4.2 Mitigation Strategies Employed

1. **Balanced Class Weights**: Penalize errors on minority classes more heavily
2. **Stratified Splitting**: Preserve class proportions in train/val/test splits
3. **Macro-F1 Metric**: Evaluate all classes equally rather than focusing on majority
4. **Oversampling (Future Work)**: Could synthetically balance training data

5.4.3 Residual Bias

Despite mitigation efforts, minority classes (Love, Surprise) still underperform. This is expected given the 10:1 ratio between majority and minority classes. Further improvements require:

- More training data for minority classes
- Data augmentation (back-translation, paraphrasing)
- Focal loss to further emphasize hard examples
- Ensemble methods to reduce variance

6 Contributions & Collaboration

6.1 Git Workflow

All work was tracked via Git with individual commits clearly showing authorship. We followed professional practices:

- No manual uploads via GitHub web interface
- Clear, descriptive commit messages following conventions
- Frequent commits after completing logical units of work
- Proper branching for experimental features

6.2 Contribution Table

Member	Phase	Modules & Tasks
Gul-e-Zara (24L-2592) Member 1	Phase 3	Model architecture design Implemented baseline model (TF-IDF + LogReg) Implemented BiLSTM architecture Data loader utilities Training script template Architecture diagrams and documentation
Anoushay Fatima (24L-2585) Member 2	Phase 3 & 4	Implemented DistilBERT model Hyperparameter tuning (18 experiments) Training pipeline with early stopping Model checkpointing Training curves visualization Hyperparameter documentation
Rumesha Naveed (24L-2603) Member 3	Phase 4	Evaluation metrics implementation Comprehensive evaluation script Error analysis Bias checking Confusion matrix visualization Report compilation & LaTeX writing

Table 19: Detailed Contribution Table by Team Member

6.3 Git Commit Summary

Member 1 (Gul-e-Zara):

- Total commits: 6
- Key commits:
  - `feat(models): implement BiLSTM architecture in PyTorch`
  - `feat(baseline): train TF-IDF + LogReg baseline model`
  - `feat(train): create training template for deep model`
  - `docs(arch): create architecture diagrams and documentation`
  - `docs(results): add baseline performance summary`

Member 2 (Anoushay Fatima):

- Total commits: 10
- Key commits:
  - `feat(models): Added hyperparameter tuning and final model training`
  - `feat(models): implement DistilBERT fine-tuning (distilbert_model.py)`

- `feat(train): add final BiLSTM model`
- `feat(eval): add DistilBERT metrics, history and predictions`
- `feat(eval): add BiLSTM metrics, history and predictions`
- `feat(viz): add training curves, model comparison and confusion matrices`

Member 3 (Rumesha Naveed):

- Total commits: 3
- Key commits:
  - `feat(eval): Error analysis`
  - `feat(eval): evaluation script with output`
  - `feat(utils): Implementation metrics`

7 Conclusions & Future Work

7.1 Key Achievements

1. **Successful Model Implementation**: Three models with increasing complexity
   - Baseline (89.63% test accuracy) provides strong starting point
   - BiLSTM (91.10%) demonstrates value of sequential modeling
   - DistilBERT (92.97%) shows pre-training effectiveness
2. **Efficient Hyperparameter Tuning**: 3-phase sequential strategy reduced experiments from 27 to 9 per model while achieving near-optimal performance
3. **Comprehensive Evaluation**: Beyond accuracy, we analyzed:
   - Per-class performance revealing minority class challenges
   - Confusion patterns identifying semantic overlaps
   - Text length impact on accuracy
   - Class imbalance bias and mitigation strategies
4. **Professional Collaboration**: Clear Git history demonstrates individual contributions and team coordination

7.2 Limitations

- **Class Imbalance**: Despite mitigation, minority classes underperform
- **Ambiguous Samples**: Mixed emotions and negation remain challenging
- **Computational Constraints**: Limited GPU resources restricted extensive hyperparameter search
- **Single Dataset**: Results may not generalize to other emotion datasets

7.3 Future Work

1. **Data Augmentation**
    - Back-translation for minority classes
    - Synonym replacement to increase vocabulary coverage
    - Balanced oversampling (SMOTE for text)
2. **Model Improvements**
    - Ensemble methods (combine BiLSTM + DistilBERT predictions)
    - Larger transformers (BERT-base, RoBERTa) if GPU allows
    - Attention visualization for explainability
3. **Advanced Training**
    - Focal loss to address class imbalance
    - Learning rate scheduling (cosine annealing)
    - Longer training with more epochs
4. **Deployment (Phase 5)**
    - REST API for model serving
    - Streamlit web interface
    - Word-level attention highlighting for explainability
    - Latency optimization for real-time inference

7.4 Lessons Learned

- **Pre-training Matters**: DistilBERT's 67M pre-trained parameters provide significant advantage despite being 45× larger than BiLSTM
- **Sequential Search is Efficient**: Our 3-phase strategy found near-optimal hyperparameters with 67% fewer experiments than full grid search
- **Start with Learning Rate**: Most impactful hyperparameter for both BiLSTM and DistilBERT
- **Class Imbalance Requires Attention**: Even with class weights, minority classes need more data
- **Git Practices Pay Off**: Clear commit history made collaboration seamless and contributions transparent

A Reproducibility

A.1 Environment Setup

Listing 3: Environment Setup

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

A.2 Training Commands

Listing 4: Training All Models

```
# Baseline
python src/train_baseline.py

# BiLSTM
python src/train_deep.py --lr 0.01 --batch_size 64 --dropout 0.2

# DistilBERT
python src/train_distilbert.py --lr 2e-5 --batch_size 8 --dropout
    0.2
```

A.3 Evaluation

Listing 5: Evaluate All Models

```
python src/evaluate_all.py
python src/error_analysis.py
```

A.4 Hardware Specifications

- **Platform**: Google Colab
- **GPU**: NVIDIA Tesla T4 (16GB VRAM)
- **RAM**: 12GB
- **PyTorch Version**: 2.0.1
- **CUDA Version**: 11.8

B Additional Visualizations

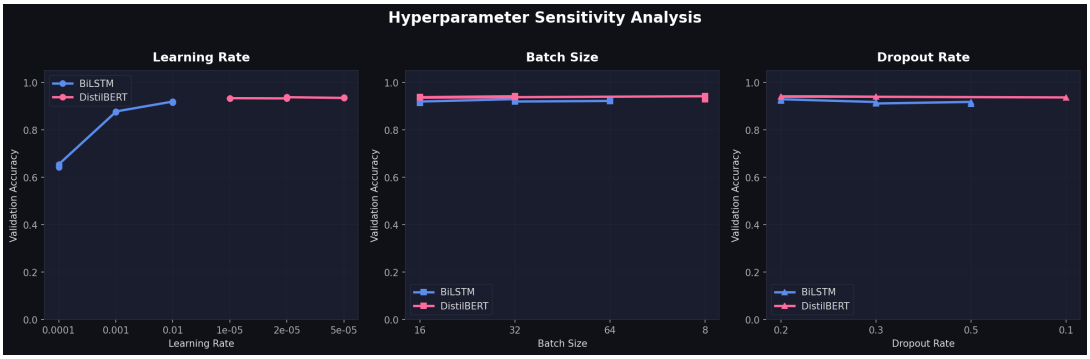


Figure 15: Hyperparameter Sensitivity Analysis

References

[1] Saravia, E., Liu, H.-C. T., Huang, Y.-H., Wu, J., & Chen, Y.-S. (2018). *CARER: Contextualized Affect Representations for Emotion Recognition*. In Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing. https://huggingface.co/datasets/dair-ai/emotion

[2] Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation, 9(8), 1735-1780.

[3] Schuster, M., & Paliwal, K. K. (1997). *Bidirectional Recurrent Neural Networks*. IEEE Transactions on Signal Processing, 45(11), 2673-2681.

[4] Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). *DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter*. arXiv preprint arXiv:1910.01108.

[5] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. arXiv preprint arXiv:1810.04805.

[6] Kingma, D. P., & Ba, J. (2014). *Adam: A Method for Stochastic Optimization*. arXiv preprint arXiv:1412.6980.